

# ch02 왜 카프카를 공부해야할까

## 왜 카프카를 공부해야 할까?

JMS, RabbitMQ와 같은 기술들이 있는데 왜 카프카를 공부해야 할까?

---

### 1. JMS, RabbitMQ, Kafka의 기본 공통점

#### 비동기 메시징

송신자와 수신자를 분리하며 **비동기 통신**이 가능하다.

#### 메시지 전달 보장

데이터 **유실 없이 안전하게 전송**이 가능하다.

#### 시스템 통합

여러 **마이크로서비스** 간 **디커플링(Decoupling)**을 통해 느슨한 결합을 구현한다.

---

### 2. 근본적 차이: 설계 철학

#### JMS / RabbitMQ - 메시지 큐

- 메시지를 **즉시 소비**시키는 메시지 큐이다
- 소비되면 **삭제**되므로 일시적이다
- **메시지 전달에 초점**을 둔다
- 전달 후 사라진다
- 수백 **TPS** 수준이며 **큐** 단위로 처리한다

**사용 예시:** 비즈니스 이벤트 알림, 비동기 작업 처리

#### Kafka - 이벤트 스트리밍 플랫폼

- 메시지를 **로그로 저장**하고 스트리밍한다
- **로그로 저장**하며 **디스크 기반**으로 재처리가 가능하다
- 데이터 **파이프라인 / 스트림** 처리에 최적화되어 있다
- **영구 보존**이 가능하다
- 수백만 **TPS** 이상 처리가 가능하며 **파티션** 단위로 순서를 보장한다

**사용 예시:** 로그 수집, 실시간 분석, 데이터 파이프라인

---

## 3. 핵심 개념 비교

### 3.1 메시지 소비

#### RabbitMQ (메시지 큐)

- 메시지가 소비되면 큐에서 사라진다

#### Kafka (로그 스트림)

- 메시지가 소비되어도 로그에 남는다

### 3.2 메시지 재처리

#### RabbitMQ (메시지 큐)

- 재처리가 불가능하다
- 수동으로 재전송이 필요하다

#### Kafka (로그 스트림)

- 메시지 재처리가 가능하다
- 오프셋을 다시 지정하면 된다

### 3.3 메시지 저장 형태

#### RabbitMQ (메시지 큐)

- 메모리 / 디스크 큐

#### Kafka (로그 스트림)

- 로그 파일 형태로 저장

### 3.4 목적

#### RabbitMQ (메시지 큐)

- 이벤트 전달

#### Kafka (로그 스트림)

- 이벤트 저장 + 스트리밍

**RabbitMQ**는 즉시 전달 후 끝난다

**Kafka**는 모든 이벤트를 로그로 남겨두고 재활용 가능하다

---

## 4. 아키텍처 차이

### 4.1 RabbitMQ 아키텍처

Producer → Queue → Consumer

- 메시지가 한 번 읽히면 **삭제**된다
- 장애 시 **복구가 어렵다**
- 단순 **알림 / 비동기 요청 처리**에 적합하다

사용 예시: 이메일 발송, 결제 승인 알림 등

### 4.2 Kafka 아키텍처

Producer → Kafka Topic(Log) → Consumer Groups

- 메시지를 토픽에 **로그 형태**로 기록한다
- **여러 컨슈머 그룹**이 동일 이벤트를 각각 독립적으로 읽을 수 있다
- **오프셋 조정**으로 과거 데이터 재처리가 가능하다
- 실시간 데이터 파이프라인, 로그 수집, 이벤트 소싱, 분석에 적합하다

**RabbitMQ:** 비즈니스 이벤트 알림용

**Kafka:** 데이터 인프라용

## 5. 결론: 왜 카프카를 배워야 하는가?

### 1) 레벨의 차이

- **JMS/RabbitMQ**는 **메시징 레벨**이다
  - 요청/응답 중심이다
  - 비즈니스 로직과 직접 연결된다
- **Kafka**는 **데이터 인프라 레벨**이다
  - 데이터 흐름 중심이다
  - 시스템 전체와 연결된다

### 2) 현대 아키텍처의 중심

- 데이터 레이크, 스트리밍 분석, 이벤트 소싱의 중심이다
- 카프카 없이는 실시간 파이프라인이 불가능하다

### 3) Spring 생태계 통합

- **Spring Boot 3.x**에서는 **spring-kafka**가 표준 메시징 모듈 수준이다
- Spring과의 강력한 통합을 제공한다

## 핵심 정리

JMS/RabbitMQ는 메시지를 전달하기 위한 큐 시스템이다

Kafka는 데이터를 저장/전송/재처리/분석까지 담당하는 스트리밍 플랫폼이다

즉, 카프카는 실시간 데이터 아키텍처의 중추 역할을 하며, 현대의 데이터 레이크, 마이크로서비스, AI 파이프라인을 공부하려면 필수적이다.

## 6. Redis로도 메시지 큐를 사용하는데 왜 Kafka를 쓸까?

### 6.1 Redis와 Kafka의 본질적 차이

#### Redis Stream

- 초고속 메모리 기반 작업 큐
- Task / Job 중심
- 작업 완료 시 삭제되며 데이터 수명이 짧다
- 시스템 범위: 서비스 내부 수준

#### Apache Kafka

- 대규모 데이터 스트리밍 플랫폼
- Event / Log 중심
- 디스크 저장 / TTL 후 삭제가 가능하다 (데이터 수명이 길다)
- 시스템 범위: 데이터 파이프라인, 시스템 전체 수준

Redis는 빠른 임시 큐로 설계되었다

Kafka는 데이터 흐름의 중추 역할을 한다

### 6.2 Redis 메시지 큐의 한계

Redis Stream이나 Pub/Sub은 전달 중심이다.

대규모 데이터 흐름의 일관성, 재처리, 확장성은 카프카가 더 강력하다.

### 6.3 왜 Redis 대신 Kafka를 써야 하는가?

#### 1) 데이터 기록

카프카는 모든 데이터를 로그 형태로 디스크에 남긴다. 메시지 전달 후 사라지는 것이 아닌 기록으로 남게 된다.

- 장애 후 복구: Consumer Offset을 다시 설정한다

- **과거 데이터 재처리**: Kafka Replay를 통해 과거 시점부터 다시 처리한다
- **실시간 + 배치 일관성 유지** 가능하다

Redis로는 불가능하다

## 2) Exactly-once에 가까운 처리 보장

Redis Stream은 기본적으로 **at-least-once**(중복 가능)이다.

Kafka는 at-least-once + **트랜잭션 기능**을 제공한다. 이는 **동일 메시지 중복 전송을 방지**할 수 있다.

데이터 **정합성이 중요한 시스템**에서 카프카가 더 안전하다.

**Exactly-once**: 데이터가 메시지 전달 과정에서 유실되거나 중복되지 않고 **정확히 한 번만 처리**된다

**At-least-once**: 메시지가 유실되지 않는 것을 보장하지만 **중복될 수 있다**

## 3) 수평 확장과 처리량

카프카는 **토픽을 파티션으로 나누고** 각 파티션이 **브로커(서버)에 분산 저장**된다.

- **서버 추가**: 처리량이 자동으로 증가한다
- **각 파티션**: 순서를 보장한다
- **복제로 장애**에도 무손실 복구가 가능하다

Redis는 단일 스트림 키에 부하가 몰리면 **직접 파티셔닝 로직을 작성**해야 하지만, Kafka는 **아키텍처적으로 확장성이 내장**되어 있다.

## 4) 다중 소비자: 여러 팀/서비스가 같은 데이터 사용

한 토픽의 데이터를 "**서비스A, 서비스B, 서비스C**" 이렇게 **여러 컨슈머 그룹**이 동일 데이터를 **독립적으로 읽을 수 있다**.

Redis는 기본적으로 **한 메시지는 한 번만 소비**하는 구조이다. 이를 구현하려면 **큐를 여러 개 만들어야** 하며 관리가 어려워진다.

## 5) 데이터 파이프라인 / 스트림 처리 생태계

카프카는 **메시지 브로커 외에도 데이터 인프라 플랫폼 역할**이 가능하다.

**통합 가능 기술**:

- Kafka Streams
- KsqlDB
- Apache Flink
- Spark Streaming
- 기타 다양한 커넥터들

Redis는 이런 생태계가 없어 **직접 코드를 작성**해야 한다.

---

## 7. 실제 예시 비교

### 7.1 Redis 사용 시

User Signup → Redis Stream → Email Worker (1개 소비자)

- 이메일 보낼 때는 괜찮다
- 나중에 가입 이벤트 기반으로 마케팅 로그를 분석하려고 할 때 이미 메시지가 사라졌다
- 재처리가 불가능하며 로그도 없다

### 7.2 Kafka 사용 시

```
User Signup → Kafka Topic: signup-event
                |
                |— Email Service
                |— Analytics Service
                |— Marketing Service
```

- 각 서비스가 독립적으로 구독한다
- 장애 시 **offset만 되돌려 재처리**한다
- 이벤트는 **일정 기간 동안 저장**된다
- 새로운 서비스 추가 시 **과거 데이터부터 재처리** 가능하다

**Redis**는 단발성 처리

**Kafka**는 지속적 데이터 스트림 파이프라인이다

## 8. 최종 결론

### Redis를 사용해야 하는 경우

- 단순 작업 큐 (이메일 전송, 비동기 작업 처리)
- 캐싱 + 메시징이 함께 필요한 경우
- **초저지연**이 필요한 경우
- 시스템 내부에서만 사용하는 경우

### Kafka를 사용해야 하는 경우

- 여러 시스템/팀이 같은 데이터를 사용하는 경우
- 데이터 재처리가 필요한 경우
- 대용량 데이터 스트리밍
- 이벤트 소싱 / **CQRS** 패턴
- 데이터 파이프라인 구축
- 실시간 분석 시스템

---

## 정리하자면

Redis는 빠른 임시 작업 큐로 훌륭하지만,  
Kafka는 데이터를 **저장하고, 흐르게 하고, 재활용하는 데이터 인프라**이다.

현대의 데이터 중심 아키텍처에서 Kafka는 선택이 아닌 **필수**이다.